



universität
uulm

**Faculty of Engineering, Com-
puter Science and Psychology**
Institute of Databases and Infor-
mation Systems

A Concept for Observability in an Internet of Things Environment

Thesis at Ulm University

Submitted by:

Claus Ruepp
claus.ruepp@uni-ulm.de
905635

Reviewers:

Prof. Dr. Manfred Reichert
Prof. Dr. Rüdiger Pryss

Supervisor:

Prof. Dr. Burkhard Hoppenstedt

2024

Version July 18, 2024

Contents

1	Introduction	2
1.1	Distributed Data Collection	2
1.2	Observability	3
1.3	Research Question	5
1.4	Structure	5
2	Related Work	6
3	Specification	11
3.1	Scope	11
3.2	Requirements	12
3.2.1	Functional	12
3.2.2	Non-Functional	13
4	Methodology	14
4.1	Abstraction of the Problem Domain	14
4.2	Analysis of Components	15
4.3	Criteria for selecting Tools	16
5	Results	21
5.1	Problem Domain	21
5.2	Component Model	23
5.3	Prospective Tools	25
5.3.1	Emitters	26
5.3.2	Communication	27
5.3.3	Storage	28
5.3.4	Visualization	30
5.4	Tool Selection	30

Contents

5.5	Final Selection of Tools	33
5.5.1	Emitter	35
5.5.2	Communication	35
5.5.3	Storage	35
5.5.4	Visualization	36
5.6	Implementation Details	36
5.7	Formalized Approach for Observability Solution	38
6	Discussion	40
7	Future Work	43
8	Conclusion	45
A	Tables	46
	Bibliography	61

Abstract

Microservice architecture software systems are growing increasingly complex. In order to maintain such complex systems, an understanding of the system's state is important. One way to gauge system state is by employing observability principles and collecting performance metrics, communication traces, and logs. Many products and tools are marketed toward this purpose, but there is no one-size-fits-all way of making use of them. This study forms an approach to implementing core principles of observability into distributed software systems. We critically analyzed the problem domain to understand the core challenges better. With the problem broken down into individual parts, we described the building blocks necessary to overcome the challenges. We then used established decision-making methods to select tools for our specific environment. We showed that our process led to a feasible solution using a proof-of-concept implementation. These methods to design our proof of concept are repeatable and form a guideline for similar problems. Our work shows that many high-quality tools are available that make laying a foundation for observability an achievable task for most projects or organizations. We did not have restrictive requirements when choosing the tools for our proof of concept implementation, limiting the significance of our findings. Another limiting factor is that we did not get a chance to gather realistic data from a live deployment. However, we are confident that we have included the means to represent different needs in our framework. We conclude that observability is within reach for any organization by following our methodical process. A basic setup can remove the barrier of entrance for further experiments and iterations.

1 Introduction

Data is a critical asset for modern businesses and individuals alike, driving informed decision-making. Data-driven decision-making supports continuous improvement initiatives. By harnessing different kinds of data, companies can gain deep insights into internal processes, resource utilization, and operational performance. These insights enable businesses to identify inefficiencies and predict failures, leading to optimized workflows, reduced costs, and enhanced productivity. In essence, data empowers us to be more agile, proactive, and responsive in a rapidly evolving landscape, ultimately leading to more sustainable growth.

1.1 Distributed Data Collection

Distributed Data Collection (DDC) is used in diverse industries and domains, whenever there is a need to collect data from multiple sources. A prime example of DDC is observability as described in section 1.2, where signals of various components of a distributed system are collected and stored centrally. Application Performance Monitoring is a derived use case, utilizing performance metrics to ensure satisfactory service for users and clients.

Internet of Things (IoT) sensor networks are networks of smart sensors that report physical measurements to central databases or data aggregators. The signals collected in sensor networks can be analyzed or visualized. In the home sector, commonplace for such IoT sensors is smart home applications. On an industrial scale, sensors can observe the health of production equipment, raw material feeds, or environmental parameters, and these inputs can be used to improve production processes, predict equipment wear and maintenance requirements, and even improve worker safety.

The agricultural sector is another industrial application that can benefit from extensive sensing. Metrics such as soil moisture and nutrient levels can be useful to optimize irrigation, and therefore improve crop yields while conserving resources.

Even in medicine, there are applications for DDC. Wearable or IoT sensors can track and collect patient vital signals and medication intake, enabling telemedical healthcare.

Finally, civil engineering can leverage traffic flow measurements, parking availability, air quality, waste management, energy consumption, and public safety in smart cities to improve urban planning, transportation, and environmental sustainability.

Sufficient and qualitative data is vital to finding adequate solutions to diverse problems, and Distributed Data Collection is how this data is acquired.

1.2 Observability

Observability stands as a crucial concept that encapsulates the ability to monitor, understand, and troubleshoot complex systems. It encompasses a range of techniques and tools that enable engineers to gain insight into the inner workings of systems, allowing them to identify and promptly resolve issues such as inefficiencies or faults.

Observability is often defined as the ability to understand the internal state of a system from external data. This means that one can infer the health, performance, and behavior of a system based on its observable outputs. The key pillars of observable outputs are as follows.

1. *Logs* provide a detailed record of system activity, including events, errors, and messages. They offer a chronological view of the behavior of the system and can be used to reconstruct the sequence of events that lead to an issue.
2. *Metrics* capture quantitative data about the system's health, such as CPU usage, memory consumption, and network traffic. They provide insights into the system's current state and trends over time.
3. *Traces* connect logs and metrics by tracking requests as they flow through the system. They provide a holistic view of the request's journey, identifying

potential bottlenecks and performance issues.

In today's world of increasingly complex and interconnected systems, observability has become indispensable. As systems become more distributed, the challenge of monitoring and understanding their behavior grows. Observability provides the means to pinpoint the root cause of performance bottlenecks, errors, and anomalies, allowing timely remediation and preventing service outages. By analyzing system metrics, engineers can identify underutilized resources and make adjustments to improve system performance, as well as efficiency, and reduce costs. Before deploying changes, engineers can use observability tools to assess their impact on the system and identify potential performance issues during development and testing.

Effective implementation of observability within a project requires a comprehensive approach that takes into account both technical and organizational aspects. On the technical side, the challenges to solve are concerned with collecting, storing, analyzing, and visualizing data. Mechanisms to collect logs, metrics, and traces need to be implemented at various points in the system. The metrics to be collected can vary depending on the point at which they are collected. At the hardware or Operating System (OS) level, data about system utilization, such as processor or memory usage, file input and output operations, or network packet ingress and egress, can be collected, while applications can expose metrics such as time-to-response or active database connections. Data storage considerations include data retention policies but also data correlation, combining per-service data on a, e.g. request basis. The stored data can then be further processed, generating alarms when system parameters reach critical thresholds or visualized in reports and dashboards, providing actionable insights into the system. Because observability is not a static state but rather an ongoing process that evolves as systems and technologies change, a culture of observability needs to be fostered within the organization, constantly emphasizing the importance of system insights. Practices and tools need to be well documented to allow engineers to maintain a deep understanding of their systems and ensure smooth operation.

1.3 Research Question

While it is apparent that observability plays a key role in enabling and managing complex software solutions today, little attention has been given to formalizing an approach for selecting technologies to fulfill this crucial role. There are many commercial solutions available, advertising immense additional value to a system. Examples include the Software-as-a-Service (SaaS) offerings of Datadog, NewRelic, or Splunk. We propose that a grounded process of analyzing the problem of observing one's system and carefully considering goals will help in choosing and implementing a fitting solution. Our main research question is therefore going to be:

- How can we effectively address the challenges of observability in distributed systems?
 - What are the core challenges of observability in a distributed system?
(1a)
 - What steps are necessary to find a suitable solution to these challenges?
(1b)

1.4 Structure

This introductory chapter has given an overview of the field of this work, outlining the relevance, and formulating our research questions. In chapter 2 we will survey existing research and literature which will shape our approach and solution, together with the scope and requirements outlined in chapter 3. Chapter 4 will detail our approach, illuminating the steps we have taken and their significance in finding appropriate answers to our research questions and delivering a solution. Chapter 5 shows our results: the problem domain is described, and a component model is presented to build a solution along with a thorough evaluation and final selection of tools for a proof of concept implementation. Our results are then interpreted in chapter 6, discussing implications, and addressing limitations. Areas that hold potential for extension and further research opportunities are outlined in chapter 7. Finally, some closing thoughts are given in chapter 8.

2 Related Work

A foundational understanding of existing research is crucial for any scientifically founded endeavor. This chapter serves as an exploration of existing literature and research on observability. We explore prior efforts to address challenges within the field, analyze their strengths and limitations, and identify potential areas for further investigation. By examining related work, we aim to build upon established knowledge and contribute novel insights to the field of observability.

The field of observability is currently undergoing a period of significant innovation and growth due to a heightened understanding of the need for introspection to be able to cope with the complexities of modern software systems, especially when employing cloud-native technologies such as orchestration and auto-scaling in conjunction with a microservice architecture [5]. A plethora of dedicated observability tools and platforms are emerging, offering features like real-time monitoring, anomaly detection, and root cause analysis. Integrating different tools, managing the ever-increasing volume of data, and ensuring cost-effectiveness are some of the ongoing concerns. Industry efforts are underway to establish common standards and best practices for observability and to ensure interoperability between tools.

Heger et al. give a quick introduction to *Application Performance Management* (APM) [7]. We are introduced to the four main activities associated with APM, which serves as a precursor for observability. The core activities mentioned are:

- Data collection: Performance measures are collected from various tiers, layers, and locations within the application system.
- Data storage and processing: The collected data is transformed into higher-level data structures for further analysis.
- Data presentation: The data is presented in a user-friendly way using different

visualizations to aid in comprehension.

- Data interpretation and use: The processed data is used to reason about the application's health, identify performance bottlenecks, and guide troubleshooting efforts.

Furthermore, the authors identify challenges for APM when applied to *microservice architecture* (MSA) systems which stem from the inherent dynamic nature of such MSA systems.

Kosinska et al. conducted a Systematic Mapping Study surveying 56 publications in the field to identify the state-of-the-art [9]. They found that there is a growing body of research on cloud-native observability, but there are still gaps in our knowledge. According to the authors, existing research is concerned with several different aspects. Motivations for equipping Cloud-native applications with observability capabilities are subject to research. Other publications survey different observability approaches, tools, and techniques used for observability. The paper also mentions that logging and tracing are underinvested areas. Furthermore, they find that cloud-native applications are complex and difficult to understand without observability. Observability helps to identify problems, improve performance, and optimize resource utilization. The authors conclude that more research is needed to develop new and improved observability techniques for cloud-native applications to be able to keep up with increasing complexity. This paper gave us a good overview of research trends and focal points within the field and made for a good entry point.

Albuquerque et al. [1] describe three proactive monitoring patterns: Liveness probes, Readiness probes, and Synthetic Testing.

- Liveness Endpoint: This pattern describes how to implement an endpoint that can be queried to check if an application is alive and responsive.
- Readiness Endpoint: This pattern describes how to implement an endpoint that can be queried to check if an application is ready to receive traffic.
- Synthetic Testing: This pattern describes how to use synthetic transactions to proactively test an application and identify potential issues before they impact real users.

They describe each pattern in detail, including its context, problem, solution, and consequences. This format helps developers understand when to apply a pattern,

what problem it solves, the trade-offs involved, and how to implement it effectively. Readiness and liveness are central concepts and probes and heartbeats are another dimension of observability data.

Cinque, Della Corte, and Pecchia [2] propose an approach to accompany microservices logs with black box tracing. The authors state that the current monitoring approaches rely on intrusive techniques that require instrumentation of all microservices. They propose a passive tracing approach that leverages request-response messages of the REST communication model to build microservices execution traces transparently. This approach does not require any modifications to the microservices themselves. Instead, it relies on an instrumented gateway which acts as a message broker between the microservices. The authors also present a case study that demonstrates the effectiveness of the proposed indirect approach in real-world scenarios.

Another study to investigate indirect monitoring is authored by Pina [12]. According to this study, the information gathered at the API gateway is sufficient to infer infrastructure topologies, service cohesion, and method or endpoint latencies without requiring the introduction of complexity into microservices in the form of monitoring instrumentation. These works on indirect monitoring and tracing shaped an understanding of the importance and capabilities of tracing data.

In their article, Waseem et al. survey the design, monitoring, and testing of microservice systems [18]. Their approach involved a survey of 106 microservices practitioners and further interviews with 6 practitioners. The survey questionnaire asked about the participants' experience with microservices, the tools and techniques they used, and the challenges they faced. The interviews provided more in-depth insights into the practices and challenges of designing, monitoring, and testing microservices systems. The most commonly surveyed metrics are resource usage, load balancing, and availability for microservices systems. Log management, exception tracking, and health checks are the most frequently employed monitoring practices for microservices systems. According to the surveyed practitioners, the most prominent challenge of observing microservice systems continues to be the collection of metrics signals, logs, and traces from containers. This article gave us an understanding of trends among practitioners and the challenges of observability.

Jiang [8] identifies a hierarchical monitoring structure that consists of multiple moni-

toring levels: Hardware, Network, Application Logs, and Application Metrics. These refer to different points within the system where signals can be collected for monitoring purposes. They also identify a need for intelligent alarm systems because existing systems often generate redundant or irrelevant alarms. An ideal monitoring solution is outlined which monitors signals from multiple monitoring levels to capture not only the state of individual instances of microservices but the system as a whole and to consider data from all levels where possible to generate alarms. We were inspired to consider multiple monitoring levels (application metrics, runtime environment metrics, host metrics) by this work, and found the inclusion of different levels to be helpful.

Some research has been done on the process of selecting metrics that are well suited to represent the state of a system which we will quickly introduce in this section.

Ram et al. explore what makes metrics actionable using a multiple case study involving four software-intensive companies [13]. An actionable metric is described as useful for guiding practitioners toward taking specific actions. The study identified that metrics that are practical, contextual, and exhibit high data quality are more likely to be actionable. Oftentimes, a metric requires interpretation to extract actionable information from it. The ease of interpretation is influenced by the complexity of the metric. Company size and project characteristics can also influence the type of metric that can be considered actionable. The authors also found that non-actionable metrics can still be useful. However, no examples of good or actionable metrics are given.

In their paper, Lomio et al. [10] propose a machine-learning approach to identify the metrics best suited to track the load of a system. The authors replicate a real-world system comprising an application server, a load generation tool, and a data collection tool. They feed the collected data from the application server as well as the number of active clients into a random forest regressor, training it on 80% of collected data and then verifying their predictions on the remaining 20% with a high R^2 score of 0.975, implying that the model has identified metrics that represent the state of the system. Metrics highlighted by the random forest algorithm are related to memory usage, network traffic, request counts, and disk activity. These identified metrics align with what is generally regarded as best practices and recommended.

Voutsas et al. [17] argue that many of the existing alerting tools flood administrators with alarms, making it difficult to identify critical alerts. This can cause alert fatigue in responsible entities, making them complacent, responding slowly, or ignoring critical alerts altogether. They propose a binary classification for alerts, identifying important alerts based on inputs such as name, category, type, value, et cetera to cut down on alerts that are oftentimes ignored anyway. This article is important because it gives a heightened understanding of the issue which is alert fatigue. This guided the decision to leave alerts to a template, avoiding this sort of fatigue to set in before custom-tailored alerting rules are ever defined.

With CloudHealth [16], Shatnawi et al. describe a system to configure monitoring probes according to clearly defined monitoring goals. Their service implements a top-down approach, starting with the definition of clear monitoring goals, which are mapped to metrics that can be collected from a software system. The CloudHealth service can be used to configure, provision, deploy, and operate the monitoring infrastructure necessary to attain the defined goals. This approach can guide the design and implementation of any monitoring infrastructure, ensuring that focus is being kept on achieving the monitoring goals.

In his 1980 book [15], Saaty describes the Analytical Hierarchy Process (AHP), a widely-used decision-making framework that has inspired various approaches. Decision-makers systematically analyze complex problems by representing them as hierarchical structures of criteria and alternatives and then evaluating pairwise comparisons to derive relative priorities. In the context of software engineering, AHP has influenced research in areas such as requirements prioritization, project selection, and software quality assessment, providing a structured and rigorous approach to decision-making that balances multiple criteria and stakeholder preferences. The AHP does not guarantee a correct decision but instead promises a decision that will fit the priorities and goals of decision-makers based on their understanding of the problem. Our decision making approach utilizes criteria to enable comparability between alternatives inspired by the AHP.

3 Specification

In this chapter, we are defining the objectives, constraints, and expectations guiding the development of the approach proposed in this thesis. We will look into the functional and non-functional requirements that guide our decisions and shape the design and implementation. First, we will define the environment that we want to observe and the motivations for implementing observability. Afterward, we will list and elucidate the requirements and minimum features for the solution.

3.1 Scope

This section outlines the environment we will observe and the most influential goals in our decision-making process. The environment we are designing our solution for is a manufacturing execution system built following the microservice architecture pattern [14]. The system is run on-premise using container virtualization via Docker. A suite of Java services forms the core of the system, with Keycloak¹ being employed to authenticate users and an API gateway to proxy the microservices. In the target system, data about production machinery is gathered by sensors and fed into the system as inputs to business logic, however, for the proof of concept, we are working with a local development environment without sensor data. Despite the complexity of the system, a small team is in charge of development and operation, making maintainability, reliability, and simplicity paramount. Our primary goals center around achieving a deeper understanding of the system to enable improvements and faster and better troubleshooting. More specifically, we want to monitor resource usage within the environment, identify long-running execution paths, and establish a centralized logging mechanism. According to [13], low-level metrics can

¹<https://www.keycloak.org/>

be sufficient indicators, however, as the complexity of an operation grows, it may become necessary to add context before they provide value.

3.2 Requirements

Next, we are defining the requirements that guided the design of our solution. Based on the problem description, we were able to extract the general sentiments that should be most influential. We represent these sentiments as requirements here. These requirements are derived from the goals, circumstances, and properties outlined above in conjunction with the problem statement and desired properties received from the responsible correspondents.

3.2.1 Functional

The solution shall enable monitoring of resource usage and utilization within the environment.

The utilization of system resources such as CPU, memory, and network is considered the best predictor of system load. Additionally, host properties such as disk utilization can enable preemptive actions to prevent unexpected service failures. This data can help plan future deployments of the product and to inform prospective clients of expected resource requirements.

The solution shall facilitate the analysis and identification of long-running execution paths within the microservices.

Trace data can inform maintainers and operators of related requests sent to the system. Latency information attached to trace data can help the search for long-running execution paths, that may be candidates for optimizations.

The solution shall establish a centralized logging mechanism to aggregate and store logs from all microservices.

When troubleshooting, the ability to correlate logs from multiple sources is immensely powerful.

3.2.2 Non-Functional

In addition to these functional requirements, more general statements have been used to describe the desired properties of the solution. These are of non-quantitative and non-functional nature.

The solution shall prioritize simplicity, ease of use, and maintainability.

Due to the small team responsible for development and operation, we want to avoid introducing complex products with much bigger scopes in favor of leaner, less complex ones. Products that can be reused to achieve multiple goals are preferred to ensure that the system does not require expert knowledge of numerous products.

The solution shall utilize free and open-source software wherever possible.

We support open-source and community-driven software projects and consider the ability to understand and maintain to be important.

Our main goal is to provide a baseline of observability functionality. These requirements further emphasize ease of use, leanness, and maintainability.

4 Methodology

This chapter details the methods used to address the research questions outlined in section 1.3. We provide a step-by-step breakdown of our approach and motivations for each step. We discuss the potential limitations and considerations associated with the chosen methodology. The goal is to provide a transparent and replicable rationale for understanding how the research questions were addressed and how the resulting findings were obtained.

4.1 Abstraction of the Problem Domain

To understand the problem well and to build a satisfying solution, it is necessary to identify the essential entities, concepts, and relationships within a specific domain to create a simplified representation that captures its fundamental characteristics. We approach this crucial step as follows:

First, we need to identify the specific problem and the goal of our work. This includes defining the constraints of the system and giving thought to the desired outcomes.

- **Identify the problem and goals**

Clearly define the specific problem we are trying to solve and consider the desired outcomes.

- **Understand the domain**

Immerse ourselves in domain knowledge, gaining insight into the concepts of the domain. This includes any sort of gathering knowledge, such as reading research papers, conducting interviews with domain experts, or studying existing systems solving similar problems, if possible. Identify the entities or

objects involved, their attributes and properties, and the relationships between them. Understand the key operations or functionalities within the domain.

- **Focus on the identified problem**

We need to ask ourselves what details are necessary to represent the core problem we identified in the first step and what further steps are necessary to achieve the goals. Filtering out irrelevant information helps to reduce complexity and focus on the necessities. Most importantly, we want to stay focused on the high-level concepts and interactions that are essential to the problem.

- **Choose appropriate abstraction and representation**

Depending on the problem at hand, different levels of abstractions and representations may be more or less well-suited. Different types of diagrams can capture different properties, and choosing the correct type can help improve clarity and keep focus on details that are relevant to the problem.

- **Refine and validate**

An effective abstraction should balance simplicity and completeness. We want to ensure that the abstraction captures the essence of the problem without losing critical details. Discussing the abstraction with domain experts or colleagues will help us validate its accuracy and completeness.

4.2 Analysis of Components

Once we have identified the entities that are part of the problem, it is important to understand their critical functionalities. For each component, we need to dive deeper and answer the question: What specific task or function does it perform? It is important to consider what data or signals the component receives as input. What is produced as output? It may also be important to consider how the component needs to function internally, i.e. what internal processes or functionalities are involved. Furthermore, we want to consider interactions and communication between components. Do the inputs and outputs of interacting components overlap? Is there a way to transform the signals (e.g. via an additional component)?

4.3 Criteria for selecting Tools

After identifying all components, we have to select a set of tools to fulfill their roles. Selecting the right tool for the project and organization is crucial to ensure that its capabilities can be leveraged. Different criteria have varying amounts of information attached to them. While a criterion like the most basic binary decision "Does this tool fulfill the requirements for the component in consideration?" can easily be answered objectively, some of the criteria, such as "Documentation" are not necessarily as straightforward. We present the main questions for each criterion and then try to give hints about how to interpret each criterion. Important criteria that we think should be considered during the selection process are as follows:

Functionality

- Does the tool meet the requirements?
This will be an objective binary question to answer. It is a necessary criterion for further consideration of the tool for the specific component role.
- Does the tool have any of the desirable optional properties?
This question is also easily answered; however, it is not necessary for a tool to also have these optional properties. Therefore, this may be seen as bonus information for some, while another might want to assign a score to this to be able to rationalize a decision more easily.
- Does the tool bring additional features?
Similarly to desirable but optional properties, this criterion can influence decision-making, albeit being hard to quantify. The answer to this can take the form of an unordered list of additional features which fall into the scope of the component. One may choose to subjectively rate these additional features based on their perceived usefulness to allow quantifying their impact on decision-making.

Maintainability

- What additional steps must be taken to maintain and ensure the functionality of the tool?

In answering this question, we are looking for information that allows a quantification of the manual labor required to maintain the tool. Questions that help in this are:

- Does old data have to be discarded manually?
- Am I required to restart regularly, that is, on a schedule?
- Is it necessary to purge caches or indices for the tool to operate reliably?

Complexity

- How much effort is required to get the product up and running?

Another rather subjective measure that incorporates factors such as internal dependencies, hardware limitations, and network conditions. Further questions to ask could include the following:

- Is the tool split into many components?
- How much knowledge of the internal workings is required to effectively use the tool?
- How much effort is required to configure it properly?

Although there are generally accepted methods of measuring the complexity of software modules on the producing side such as *cyclomatic (McCabe) complexity* [11] and *Halstead measures* [6], there does not appear to be such an objective measure of operational complexity. Instead, we must once again rely on subjective assessments to incorporate this aspect into our decision-making process.

Dependencies

- Does the tool depend on any necessary scaffolding?

A tool that has a hard requirement for a certain infrastructure such as databases or message brokers should be considered with the incurred additional debts in mind. It may be wise to evaluate these dependencies using the same criteria as in this catalog.

Performance

- Are there any potential bottlenecks due to the use of the tool?

Where possible, data from tailored benchmarks would be ideal for an objective comparison. In cases where the operation is as yet small enough to be operational without worrying about performance issues, this measure can lead into considerations about the future-proofness of a particular tool.

Cost

- What are the costs associated with employing the tool?

Consider upfront and operating costs for licensing, infrastructure, or access. Maintenance and upkeep are to be considered, too, and cost is therefore further influenced by complexity.

Documentation

- How well is the tool documented?

A subjective rating can be formed based on categories ranging from well-established open-source project down to no documentation at all. A well-documented but complex product may be more usable than a simpler one with zero documentation.

Support

- Is support available? In what form? (Community, Ticket system, Service Level Agreements, et cetera)

While there is a binary aspect to this question, different support modalities may be preferred within your organization and therefore can impact decision-making.

Community

- Do forums exist?

- How active are these forums?

Consider employee versus user activity. An active community can point towards a satisfying product. Community involvement can be further gauged by surveying projects that interact with the tool and their user base.

Security

- What authentication mechanisms are available?
- Are secure communication channels supported?
- Is data stored in an encrypted format?

Security requirements differ between organizations, so this criterion merits different approaches to rating and incorporating it into decision-making. Due to these varying requirements, the main takeaway here should either be

- "Are my security requirements fulfilled?",
which can have a binary influence similar to the first criterion of functionality, or
- "What are supported security mechanisms?"
which would yield a list of features.

Reliability

- How high is the chance of breakage?
- Is the product tolerant to faults?

Unfortunately, it is nearly impossible to claim the reliability and stability of a complex system, especially due to external influences. It is, however, possible to gauge something as complex as reliability by surveying community feedback and support forums. Sometimes, the answer may even lie in the approach taken by the producer of a tool:

- Are they implementing their own solutions to complex tasks or relying on proven and established solutions instead?

With these criteria, we hope not only to gauge suitability on a functional level but also to bring quality and support into the equation.

In conclusion, we are applying a divide-and-conquer approach, transforming our high-level problem into several approachable sub-problems. We are relying on critical and analytical thinking to transform the problem statement into a modular representation. Based on the modular representation we can build on established methods to select tools that correspond with our needs. A catalog of criteria (section 4.3) helps us consider all aspects of prospective tools. In the next chapter, we will present our (intermediary) results of applying the methods described.

5 Results

This chapter presents the key findings obtained from applying the methodological approach outlined in chapter 4. The research questions outlined in chapter 1 serve as the guiding principles for this approach.

5.1 Problem Domain

To effectively deliver a suitable solution, we must gain an in-depth understanding of the problem domain. We will follow the procedure outlined in section 4.1 to arrive at a reasonable abstraction of the problem, that we can utilize as a basis for further decisions. The diagrams presented are refined and a product of iteration.

Identify the problem and goal We want to generate, store, and visualize information about a system made up of Java services as well as further tools, such as authentication provider Keycloak and relational database system MariaDB. Our goal is to gain an improved understanding of system behavior concerning resource consumption, service interconnections, and root cause identification.

Understand the domain This step involves researching and learning about the problem domain. Our understanding of the problem domain stems from the work presented in chapter 2, experience, and research on products that are advertised as solutions within the domain. An overview of our understanding of the problem is visualized in fig. 5.1. This draft showcases the problem domain at a high level of abstraction.

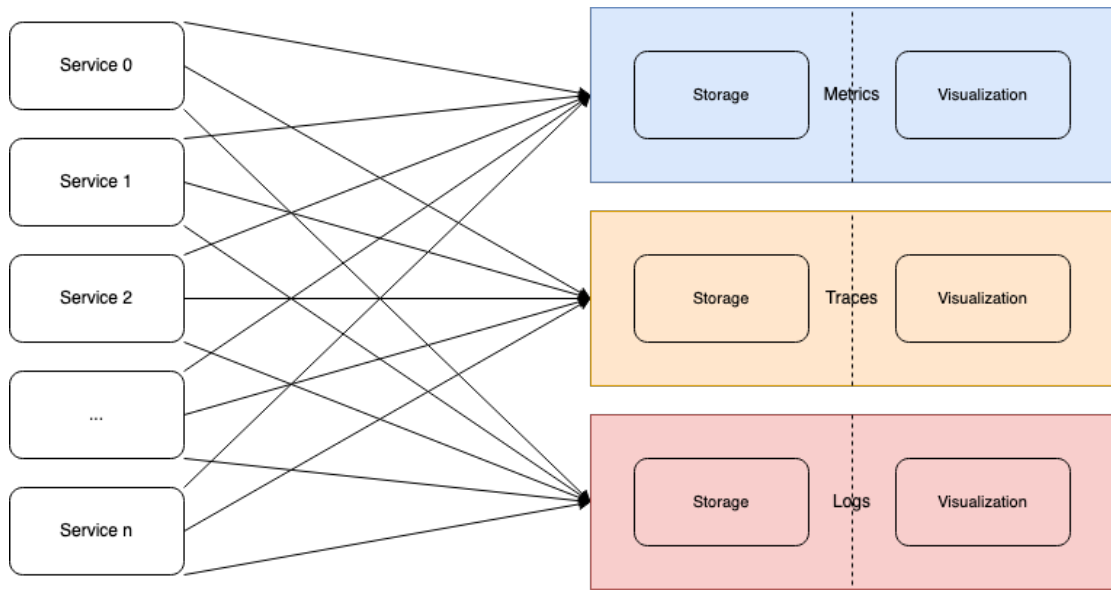


Figure 5.1: Problem Domain Overview

Focus on the identified problem The core problems we have identified can be better represented when drilling down from the overview presented in the previous paragraph. For each type of signal, there are several necessary conditions.

First, the signal needs to be generated.

A mechanism is required to store and access signals.

A mechanism is required to access, process, analyze, and visualize stored signals.

The signal needs to be transported from the component where it is generated to storage.

The signal needs to be generated in a format that can either be processed by the storage component directly or be transformed into a format that can be processed.

These challenges are represented in fig. 5.2.

Choose appropriate abstraction and representation Since the essence of the problem can be described as selecting and connecting building blocks, we choose to visualize using domain diagrams. Domain diagrams are commonly used in *Domain Driven Design* (DDD) [4]. DDD is a software engineering design paradigm focused on representing real-world problem domains with software components, mimicking interactions between entities through object methods.

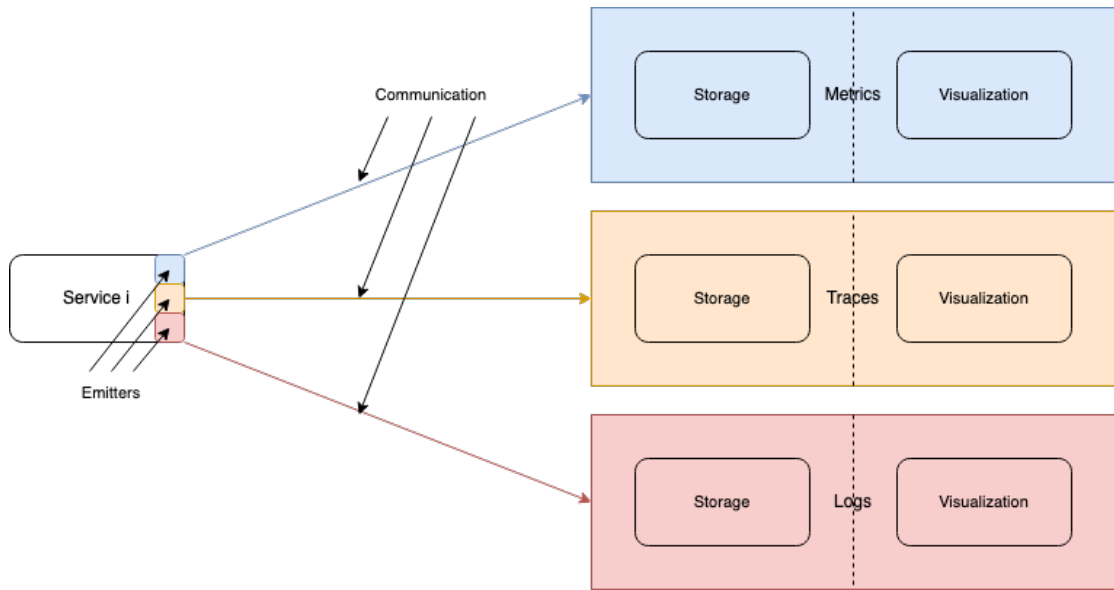


Figure 5.2: Component Detail View

Refine and validate We validated our understanding and representation of the problem domain with engineers at the company that we worked with.

Our refined problem domain model serves as the foundation for further steps. It showcases our understanding of the participating components: Emitters, Communication Layer, Storage, and Visualization components. The figure also shows the interconnections and relationships between these entities, which we will further define in section 5.2.

5.2 Component Model

In this section, we are going to examine each component further. For each component, we need to identify what specific task or function is expected, therefore serving as a qualifying or necessary property. Furthermore, each component can have additional properties that may not be necessary for qualification but can generally be considered desirable. An example of this could be support for horizontal scaling: horizontal scaling may not be immediately necessary for a signal storage component if the volume of signals that are generated by a system is low. How-

ever, systems can evolve, and with evolution, the amount of signals generated and ingested may increase drastically. Scalability is therefore desirable because it promotes future-proofness.

Emitters Emitters are responsible for generating and making a specific type of signal available. Most commonly, emitters will be instrumentation that is integrated into software modules or the execution environment. For an emitter, a required property is therefore the ability to generate observable signals from the state or activity of the component it is attached to. A desirable property in an emitter is a low implementation overhead, i.e. the possibility to instrument a component without extensive code changes.

Communication We use the term "communication" broadly to refer to any protocol or additional software component that is required to facilitate the transport of signals generated by an emitter to signal storage. The facilitation of transport is the main requirement for a viable technology. Furthermore, we would like to avoid adding more services into the environment wherever possible to keep the maintenance overhead down.

Storage The storage component is a purpose-built, database-like system with specialized tooling to ingest signals. The main requirement for the storage component is persistent storage, but a query interface is also necessary to access stored signals. Given that not every type of signal necessarily requires its own storage backend, our preference is a reusable solution capable of storing e.g. metrics as well as traces to keep the number of employed components to a minimum. Similarly, a low maintenance overhead is desirable for a storage component. While not initially critical, it would also be welcome if the product could be scaled horizontally, to allow its use in higher throughput scenarios.

Visualization These tools leverage the query interfaces provided by the storage systems to extract and display valuable insights and alerts derived from the stored signals. Key properties of this component are seamless interaction with the storage systems and the capability to define visualizations that present the extracted

Component	Description
Emitters	Instrumentation that is added to the services and emits signals. NP: Generate signals, make signals available DP: No or little changes to service
Communication	Protocols and auxiliary components to allow transmission of signals into signal storage. NP: Support for protocol on both sides DP: No additional components necessary, low overhead
Storage	Database-like systems that store signals and allow querying for stored signals. NP: Persistent storage DP: Reusable, low maintenance overhead, scalable
Visualization	Tools that employ the query interfaces of the storage systems to allow the displaying of various insights or alerts generated from stored signals. NP: Compatibility with storage system, capability to visualize as required DP: Reusable, low maintenance overhead

Table 5.1: Tool Selection Criteria Overview

insights and alerts in a user-friendly and customizable format. This could involve charts, graphs, tables, or other visual representations tailored to user needs. While not mandatory, some additional properties we are looking for in a visualization tool are reusability and a low maintenance overhead. If the tool can interface with multiple storage systems with minimal modifications, time and effort in configuring multiple tools for different signal types can be saved, and users will be allowed to quickly grasp patterns, trends, and anomalies across several aspects of the surveyed system.

Table 5.1 summarizes the necessary (NP) and desirable (DP) properties and gives a short overview of each component.

5.3 Prospective Tools

Before assessing and selecting tools, we are collecting a set of candidates for each component role. This list is non-exhaustive; it includes only candidates that appear

compatible with our given general requirements and simultaneously capable of fulfilling our monitoring goals. We include URLs to landing pages or entry points for each project or tool to disambiguate.

5.3.1 Emitters

Micrometer Micrometer¹ is a metrics instrumentation library for Java applications, providing a simple and consistent API for monitoring application performance and health. It simplifies the process of instrumenting Java applications for monitoring and provides a flexible and powerful framework for collecting, managing, and analyzing application metrics across different monitoring systems.

OpenTelemetry OpenTelemetry² (OTEL) is a vendor-neutral observability framework that standardizes the collection of telemetry data, including traces, metrics, and logs, across distributed systems. It provides a unified API and instrumentation libraries for popular programming languages and frameworks, enabling developers to instrument their applications for monitoring and troubleshooting purposes. With its flexibility and extensibility, OpenTelemetry allows users to integrate with various monitoring systems and platforms, such as Prometheus, Jaeger, and Grafana, to gain insights into application performance and behavior. By promoting interoperability and collaboration within the observability ecosystem, OpenTelemetry aims to simplify the process of instrumenting, collecting, and analyzing telemetry data in modern, cloud-native environments. For the emitter component role in particular, we consider the OpenTelemetry Agent which is a Java Agent that supports auto-instrumentation of many popular Java frameworks including Spring Boot.

Another piece of the OpenTelemetry ecosystem which we consider in section 5.3.2 is the OpenTelemetry Collector, which receives, processes, and exports data from and to various other software, enabling seamless communication.

Docker Stats The Docker daemon³ exposes a stats endpoint for retrieving real-time resource usage metrics of Docker containers and the Docker daemon itself.

¹<https://micrometer.io/>

²<https://opentelemetry.io/>

³<https://www.docker.com/>

This endpoint provides insights into CPU, memory, network, and disk I/O usage of running containers, facilitating comprehensive monitoring and troubleshooting. By leveraging the daemon endpoint, users can access key metrics programmatically, enabling integration with monitoring systems and automation tools for streamlined container management.

5.3.2 Communication

We will focus on software that can facilitate communication through translation of messages between components.

OpenTelemetry Collector The OpenTelemetry Collector⁴ is a flexible and scalable component that streamlines the collecting, processing, and exporting of telemetry data across distributed systems. It aggregates data from diverse sources, including traces, metrics, and logs, providing a unified platform for observability data management. With its customizable pipeline architecture, the collector supports various data processing tasks such as filtering, transformation, and enrichment, enabling users to tailor telemetry data to their specific needs. By offering seamless integration with popular observability backends and protocols, the OpenTelemetry Collector empowers organizations to centralize and standardize telemetry data collection, facilitating comprehensive insights and analysis into system performance and behavior.

Telegraf Telegraf⁵ is a versatile and lightweight agent made by InfluxData for collecting and reporting metrics data from various sources. It supports a wide range of input plugins for collecting metrics from systems, services, and applications. Telegraf's modular design allows users to easily configure and extend its functionality by enabling or disabling plugins according to their monitoring requirements. With its output plugins, Telegraf can forward collected metrics to numerous monitoring systems and time-series databases, facilitating seamless integration.

⁴<https://opentelemetry.io/docs/collector/>

⁵<https://www.influxdata.com/time-series-platform/telegraf/>

5.3.3 Storage

Metrics

Prometheus Prometheus⁶ is an open-source monitoring and alerting toolkit designed for reliability, scalability, and observability in modern, cloud-native environments. It collects time-series data through a pull-based model, scraping metrics from instrumented applications and infrastructure components at regular intervals. With its powerful querying language, flexible alerting system, and rich ecosystem of integrations, Prometheus enables insights into system performance, anomaly detection, and proactive incident responses.

InfluxDB v2 InfluxDB v2⁷ is a versatile and scalable time-series database built for handling high volumes of time-stamped data from diverse sources. It offers a comprehensive platform for storing, querying, and visualizing time-series data. With its powerful query language, flexible data models, and built-in visualization tools, InfluxDB v2 provides a robust foundation for monitoring, analytics, and real-time decision-making in modern environments.

InfluxDB v3 InfluxDB v3⁸ builds upon the strengths of its predecessor, offering enhanced performance, scalability, and usability for handling time-series data at scale. With improvements in storage efficiency, query execution speed, and management capabilities, InfluxDB v3 provides an even more powerful platform. The main selling point for v3 for our use case is its ability to store trace signals as well as metrics signals. Unfortunately, as of the time of testing these tools, there was not a sufficiently stable version of InfluxDB v3 available, so it was left out of further consideration.

Traces

⁶<https://prometheus.io/>

⁷<https://docs.influxdata.com/influxdb/v2/>

⁸<https://www.influxdata.com/products/influxdb-overview/>

Grafana Tempo Grafana Tempo⁹ is a distributed tracing backend designed to simplify observability in distributed systems. It offers an effective and scalable solution for collecting, storing, and analyzing trace data, supporting the OpenTelemetry standard for instrumentation. With its focus on simplicity, scalability, and compatibility with other monitoring tools, Grafana Tempo aligns well with our goals.

Jaeger Jaeger¹⁰ is an open-source, end-to-end distributed tracing system designed to monitor and troubleshoot complex microservices architectures. It provides deep visibility into the flow of requests and responses across service boundaries, enabling users to diagnose performance issues and optimize system reliability. With its scalable architecture, rich visualization capabilities, and seamless integration with popular observability tools, Jaeger facilitates comprehensive tracing and analysis.

Zipkin Zipkin¹¹ is an open-source distributed tracing system designed to assist developers in understanding the complexities of microservices architectures. While it has historically been a popular choice, it has seen decreased adoption in favor of more modern solutions like Jaeger. Despite this trend, Zipkin continues to offer valuable tracing capabilities, providing insights into request flows and service dependencies for users seeking a lightweight, familiar tracing solution.

Logs

Elasticsearch Elasticsearch¹² is an expansive, distributed search and analytics engine renowned for its scalability and real-time processing capabilities. With a rich feature set, including advanced search queries, data indexing, and aggregation, it serves as a comprehensive solution for managing and analyzing large volumes of data. However, its complexity and resource requirements may be daunting for smaller-scale projects or organizations with resource restrictions.

⁹<https://grafana.com/oss/tempo/>

¹⁰<https://www.jaegertracing.io/>

¹¹<https://zipkin.io/>

¹²<https://www.elastic.co/elasticsearch>

Grafana Loki Grafana Loki¹³ is a horizontally scalable log aggregation system designed for storing, exploring, and analyzing logs from cloud-native environments. Built for simplicity and efficiency, Loki offers a cost-effective solution for collecting and querying logs, utilizing a highly efficient indexing strategy. Its flexible architecture allows seamless integration with other logging and monitoring tools, facilitating streamlined log analysis and troubleshooting workflows.

5.3.4 Visualization

Grafana Grafana¹⁴ is an open-source visualization tool that connects to countless data sources to allow users to gain insights into their data and the systems that are represented by that data. Its main feature is configurable dashboards, which allow combining different visualizations, including tables, graphs, gauges, and more. The visualizations are backed by data retrieved from user-defined queries to data sources and can be further configured to suit the user's needs. Grafana also offers configurable alerting rules.

Two of Grafana's competitors are Kibana and Perses. However, Kibana¹⁵ only connects to Elasticsearch data sources and Perses¹⁶ currently only supports Prometheus data sources, locking us into certain picks for other components while only accomplishing visualization for some of the signal types. Other options such as SigNoz¹⁷ or Datadog¹⁸ are branded "Observability as a Service" and aim to solve most concerns of observability rather than just data visualization and can therefore hardly be considered alternatives.

5.4 Tool Selection

In the next step, we apply our selection criteria to each candidate. This section will showcase the considerations for different criteria and components. We choose

¹³<https://grafana.com/oss/loki/>

¹⁴<https://grafana.com/grafana/>

¹⁵<https://www.elastic.co/kibana>

¹⁶<https://github.com/perses/perses>

¹⁷<https://signoz.io/>

¹⁸<https://www.datadoghq.com/>

InfluxDB OSS v2 as an exemplar, considering it for use as metrics signal storage. The signal storage component is discrete and contained, making it a good choice for illustration purposes. Each aspect is considered to form an opinion about the tool, its ecosystem, and its suitability for the component role.

Functionality The necessary property for a metrics storage component is persistent storage of metrics signals. InfluxDB OSS v2 has this capability and is therefore a viable product. As for desirable properties, this version of InfluxDB can be reused within the environment as signal storage for IoT sensor signals. Furthermore, we consider the maintenance overhead to be low, however, scalability is not available in the free open-source software (OSS) variant that is being considered.

Extra Features InfluxDB OSS v2 offers several additional features which can be helpful. Most notably, a web user interface is offered with access to functionality such as a data explorer, query builder, and alert manager.

Maintainability Features such as configurable data retention periods help with maintainability. According to the documentation, it is recommended to flush time-series indices periodically to purge corrupt or out-of-date caches.

Dependencies InfluxDB OSS v2 does not have hard dependencies on other software, however, use in conjunction with InfluxData's own Telegraf application is recommended. Telegraf can ingest and transform data for InfluxDB to increase compatibility with data sources and emitters, but it is optional.

Complexity The OSS variant of InfluxDB is available as a single binary and runs as a single container. Many configuration options are available, but it can run well out of the box. Running or managing the database does not require in-depth knowledge of its internal workings, making it relatively straightforward and user-friendly.

Performance For our use case, performance is not highly critical due to the extent of the environment. In general terms, InfluxDB appears to be faster than

NoSQL databases such as ElasticSearch, Cassandra, or MongoDB; however, the lack of horizontal scalability in the free OSS version may limit this to smaller workloads. Other time series databases such as TimescaleDB are faster at ingesting and querying on average. Due to its data format, InfluxDB is very space-efficient. Overall, the performance of InfluxDB is not a risk for our scenario.

Cost InfluxDB OSS v2 is free and open-source software. There is paid support available as well as a managed service offering.

Documentation InfluxData offers extensive documentation of both the configuration options as well as query interfaces. More advanced aspects such as architecture and design are documented as well.

Ecosystem On top of many official libraries and integrations, there are a host of community-contributed adapters, extensions, and integrations with other products available.

Community InfluxData offers several channels to interact with other users and employees. There are official forums and an official Slack channel available to share and discuss with other users as well as employees. The GitHub repository has nearly 28k stars and over 500 contributors, indicating a lively community and plenty of users.

Support Next to the official forums and Slack, support for the OSS version of InfluxDB can be found in GitHub issues of the project. Furthermore, paid support is available as well.

Security InfluxDB offers support for TLS to secure communications. Clients can further be authenticated and granted access through the use of API tokens.

Reliability Unlike some competitors in the space of time series databases that rely on established database systems such as PostgreSQL for an easy solution to problems like data storage and integrity, InfluxDB is built from the ground up. While this approach gives the architects the highest degrees of freedom, it also means a lot of effort will have to be put into solving complex problems that have been solved and tested in other projects. There are reports of inconsistent or corrupt data from InfluxDB users, so this appears to be a known issue. Reports of fully non-functional databases are few and typically seem connected to user errors or external influences. Since we are not planning to store highly critical data in InfluxDB, the risk of data loss or corruption due to faults can be considered acceptable.

All these considerations are condensed in table A.8 to offer a glanceable and more easily comparable format. Each criterion is rated using a subjective scale. Given the same research effort for each system, a basis can be formed for selecting from a variety of tools. As for InfluxDB, we consider it a viable option to store metrics signals because of its ease of use and reusability as a store for IoT signals elsewhere in the overall system.

Tables assessing InfluxDB v2 and all other alternatives can be found in appendix A.

The ratings from all assessed tools are shown in table 5.2 for easier comparability. We assigned ratings based on subjective assessments, ranging from 0 (very negative) to 4 (very positive).

5.5 Final Selection of Tools

Finally, we need to select a tool for each component role. As pointed out in section 4.3, the selection can be based on subjective or objective criteria defined by requirements. Since our requirements are not strictly defined performance characteristics but general directions such as maintainability and reusability, our criteria are not assigned ratings on a scale based on evidence and a degree of subjectivity will be involved.

Tool Type	Micrometer Emitter	OpenTelemetry Agent Emitter	Docker Stats Emitter	OpenTelemetry Collector Communication	Telegraf Communication	Plain Communication	Grafana Visualization
Necessary Properties	4	4	2	4	4	2	4
Desirable Properties	2	3	4	2	2	4	4
Extra Features	2	3	2	4	4	2	3
Dependencies	2	2	4	4	4	4	4
Complexity	1	3	4	3	3	4	4
Maintainability	2	2	4	4	4	4	4
Performance	2	2	4	2	2	4	4
Cost	4	4	4	4	4	4	4
Documentation	4	3	4	3	4	2	4
Ecosystem	3	4	2	4	4	2	4
Community	3	3	4	4	4	2	4
Support	4	4	2	4	4	2	4
Security	2	4	2	4	2	2	4
Reliability	2	2	2	2	2	2	4
Sum	37	43	44	48	47	40	55

Tool Type	Prometheus Metrics Storage	InfluxDB v2 Metrics Storage	InfluxDB v3 Metrics Storage	Jaeger Trace Storage	Grafana Tempo Trace Storage	Zipkin Trace Storage	Grafana Loki Logs Storage	Elasticsearch Logs Storage
Necessary Properties	4	4	4	4	4	4	4	4
Desirable Properties	2	3	3	3	3	3	3	2
Extra Features	3	4	4	3	2	2	2	3
Dependencies	4	4	4	3	4	0	3	1
Complexity	4	4	0	3	4	4	4	1
Maintainability	3	3	1	3	4	4	4	1
Performance	4	3	4	3	4	3	4	4
Cost	4	4	4	4	4	4	4	3
Documentation	4	4	4	3	4	4	4	4
Ecosystem	4	3	3	4	4	3	4	4
Community	4	4	2	4	3	2	4	4
Support	4	3	4	3	4	2	4	4
Security	4	4	4	2	4	1	3	4
Reliability	4	1	2	4	4	4	4	4
Sum	52	48	43	46	52	40	51	43

Table 5.2: Overview table of subjective ratings used as basis for final tool selection. Ratings range from 0 (very negative) to 4 (very positive).

5.5.1 Emitter

For instrumentation within the Spring Boot Java services, we have decided to forego Micrometer in favor of the OpenTelemetry Java Agent. The OTEL Agent does not require changes to the services' dependencies and can be plugged into existing container configurations seamlessly. On top of the OTEL Agent, we have also opted to collect Docker stats and host signals to gain additional insights into the state of the services and the Docker host. These signals are periodically scraped by the OpenTelemetry Collector.

5.5.2 Communication

We chose to integrate the OpenTelemetry Collector into our solution to take advantage of its pre-processing and batching capabilities. Additionally, it moves the responsibility of talking to storage backends out of the Java services and allows us to essentially hot-swap storage backends without interrupting the operation of the services. Thanks to its highly configurable and extensible design, the OpenTelemetry Collector can also be used to collect signals that are offered in a pull configuration (such as Docker and host stats) and write them to push-based databases.

5.5.3 Storage

Metrics One of our objectives was reusability, keeping down the number of components requiring maintenance. Since InfluxDB v2 was already part of the target environment, and version 3, which additionally supports trace signals, was not available at the time, we opted to use version 2. The concerns we have outlined in section 5.4 are outweighed by its reusability and ease of use.

Traces For trace signal storage, we chose Grafana Tempo over Jaeger despite Jaeger's status and regard in the industry. While both support an all-in-one configuration that does not require additional software for persistent storage, Tempo's design and reliance on all-purpose storage rather than a high-complexity database system for a scalable setup is more appealing. Grafana Tempo also integrates nicely with Grafana and Grafana Loki, if chosen to leverage.

Logs Since we were looking for simplicity and both Grafana Tempo and Loki follow very similar approaches, we picked Loki and its companion Promtail to ship and store logs over Elasticsearch. Elasticsearch's Github description [3] reads "[...] optimized for speed and relevance on production-scale workloads", targeting large-scale deployments and simultaneously implying a scope mismatch with our requirements.

5.5.4 Visualization

Grafana is considered the de facto standard for data visualization from a plethora of data sources and is very robust so we consider it a safe pick. Additionally, no self-hostable, visualization solution with similar support for data sources is available.

In summary, our solution will include an instance of the OpenTelemetry Agent attached to all the Java services emitting trace and metrics signals to the OpenTelemetry Collector while Promtail scrapes application logs and forwards them directly to Grafana Loki. The OpenTelemetry Collector fulfills two roles: forwarding received signals to their respective storage backends and pulling information from the Docker daemon stats endpoint and the host file system. The three storage backends are InfluxDB v2 for metrics; Grafana Tempo for Traces; and Grafana Loki for Logs. Grafana is used as the user interface for querying data from all storage backends. Grafana supports user-defined visualizations and alerts, serving as a central interface to explore all types of collected data.

5.6 Implementation Details

Figure 5.3 illustrates the final architecture of our observability solution implemented for a distributed system comprising multiple Java services. We integrate the open-source tools we previously selected to collect, process, store, and visualize metrics, logs, and traces.

The observed system is represented as Java services $0..n$, which are based on the popular Spring Boot framework. These services form the core of the distributed

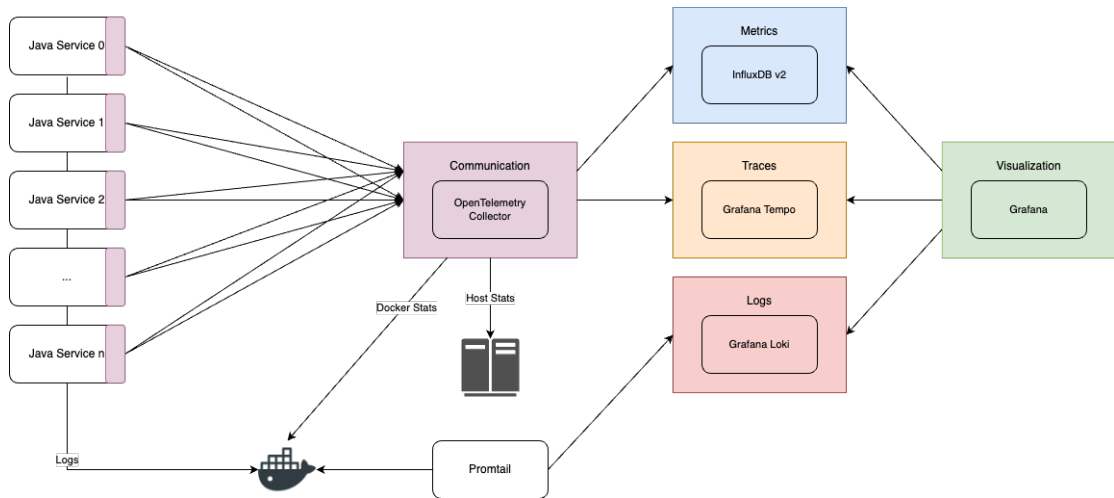


Figure 5.3: An overview of the final setup

application. We instrument the services via the OpenTelemetry Java Agent, which leverages the Java Instrumentation API to alter the byte code of commonly used libraries such as Spring Boot to emit metrics and trace signals. This approach does not require changes to the source code or build process of the services while enabling a broad baseline of signals to be collected.

Standard output and standard error log streams are already managed by the container runtime, Docker. Promtail is deployed to collect logs from the Java services. It acts as an agent that reads log files and forwards them to Grafana Loki, ensuring that all logs generated by the Java services are captured and sent to the logging backend.

The OpenTelemetry Collector serves as the central communication hub for metrics and traces. It receives telemetry data from the instrumented Java services and other sources, pre-processes it, and then exports it to the appropriate storage backends. The collector supports a variety of data formats, import, and export protocols, ensuring that all components can communicate. On top of its role as a communications hub, the collector scrapes metrics from the container environment and the host system. In this way, we can gain more context when investigating or troubleshooting.

The OpenTelemetry Collector forwards signals to respective storage backends: InfluxDB v2 for metrics, Grafana Tempo for traces, and Grafana Loki for log storage. InfluxDB stores metrics data efficiently and allows for the querying of time-series

data. Grafana Tempo collects trace data, enabling the tracing of requests across different services. Grafana Loki indexes incoming log lines for efficient querying and stores them in a compressed format.

Grafana serves as the unified visualization layer. It integrates with the storage backends to query metrics, traces, and logs centrally. Grafana allows users to create custom dashboards, set up alerts, and perform in-depth analysis of the collected observability data.

In summary, this architecture employs a modular approach to observability, leveraging Promtail, OpenTelemetry Collector, InfluxDB v2, Grafana Tempo, Grafana Loki, and Grafana to achieve comprehensive monitoring and analysis of a distributed system. Each component plays a crucial role in ensuring that logs, metrics, and traces are efficiently collected, processed, stored, and visualized, enabling better understanding of the system's behavior.

5.7 Formalized Approach for Observability Solution

The results presented in this chapter demonstrate that our approach is effective at building a baseline solution for observability in a distributed software system. The modular representation of the problem space enabled a structured approach. We proposed component criteria that can aid the analytical selection of tools for an implementation. Finally, our proof of concept cemented the feasibility of this approach. We have therefore established a foundation for further development and real-world application of observability. Overall, we consider our process to form a framework for solving high-level problems of this type, which we propose as an additional contribution of this work. We acknowledge that this is only applicable on a high level and does not address more specific questions that may arise in different contexts.

As we transition to the discussion chapter, we will delve deeper into the interpretation of these results, exploring their practical and theoretical implications. We will also address the limitations identified during the research, such as the small-scale development setup and general requirements, and suggest directions for future research to build upon this work. Overall, the results provide a promising starting

point for developing comprehensive observability solutions in complex distributed environments.

6 Discussion

This chapter explores the challenges of implementing observability principles in distributed software systems and presents our findings. Our main goal was to answer this question:

- How can we effectively address the challenges of observability in distributed systems?

To achieve this, we break down the problem further with two sub-questions:

- What are the core challenges of observability in a distributed system? (1a)
- What steps are necessary to find a suitable solution to these challenges? (1b)

Our research examined various methodologies for addressing the observability problem. The key finding is the development of a methodical approach that facilitates the integration of observability principles into distributed software systems. This approach utilizes a modular representation of the problem space, making the process of finding a solution more manageable and scalable. The effectiveness of this approach was validated through the implementation of a proof-of-concept system. Our approach closely resembles Domain Driven Design. Similarly to DDD, we have created a model of the problem domain by applying expert knowledge, and then chosen components (rather than implementing them ourselves) that interact akin to the model.

The results are promising, demonstrating the ease of implementing observability in distributed systems. We addressed the first sub-question through the modular representation of the problem domain. This divides the problem into smaller parts and enables a divide-and-conquer approach. Furthermore, our research revealed extensive existing knowledge and numerous tools available to overcome the core

challenges we identified: signal generation, communication, storage, and visualization. We acknowledge, that at larger scales, solving communication and storage becomes increasingly complex due to the volume of signals.

The answer to research question 1b lies in the process we have presented. The steps we took are inspired by tried and proven methods such as Domain Driven Design and the Analytical Hierarchy Process for solving complex decision-finding problems and founded on knowledge from research in the field. We provide a guideline that emphasizes staying focused and within scope and verified the viability of our methods through a proof of concept.

Our criteria catalog for selecting software proved helpful by ensuring we considered not only the functional aspects of each product. We assigned subjective ratings on a scale of 0 to 4 to each criterion. Ratings of 0 were treated as *very negative* impressions, 2 is a *neutral* score and 4 is *very positive*. Table 5.2 offers an overview of all the ratings we have assigned, with most of those ratings being at least neutral or positive. This positive bias can indicate the overall quality of the products we considered. We think defining scales or assessment hints for each criterion could improve the performance and make the decisions more transparent. Objective criteria such as GitHub stars were especially easy to fill out. At the same time, performance assessments often had to rely on anecdotal evidence rather than benchmarks due to a lack of data. Additionally, adjusting scales via weights can be helpful in better representing the needs of a project or organization. One example of this being important is the metrics storage component, where we chose to employ InfluxDB rather than Prometheus despite our assessments of both tools. This is due to InfluxDB being used to collect sensor data already. The criteria failed to reflect this, but we consider this particular case an outlier.

It is important to acknowledge the limitations of the work we did. We conducted our work within a small-scale development environment with general and relaxed requirements. The data we gathered therefore only extends to the proof-of-concept level. While the findings demonstrate how easy it is to build the foundation for an observability solution, they have not been tested in real-world scenarios with substantial data volumes or complex constraints. Our assessments were based on subjective and potentially shallow evaluations and impressions of complex tools.

To address these limitations, future work should:

- Define more specific and measurable requirements and goals for implementing observability.
- Develop a concrete plan for deploying the solution in at least a production-like environment to assess its effectiveness in a practical setting.
- Gather and analyze data beyond the initial proof-of-concept stage to gain insights into the solution's scalability and performance under realistic workloads.
- Define discrete scales for each criterion and incorporate weights to represent needs better.

By addressing these limitations and incorporating the suggested improvements, future work can build upon the foundation established in this research and provide an improved, more comprehensive, and robust approach to observability.

7 Future Work

In this chapter, we are considering some of the trends and possibilities for further work within the field of observability.

Troubleshooting failures after the fact can be considered the most naive use case for logs. The practices of log aggregation and log mining can be used to extract additional value from information that is already being collected. Our solution focuses on aggregation, transporting logs from distributed sources, storing them centrally, and indexing them for faster keyword searches. Log mining is an advanced concept applied on top of log aggregation, and describes the process of log analysis to extract valuable insights from logs beyond their usefulness for troubleshooting. Aggregated logs need to be preprocessed before further analysis is possible, normalizing information, and extracting features like timestamps, event types, IP addresses, and many more from log messages. Once normalized, data mining operations such as clustering, classification, association rule mining, anomaly detection, and sequence mining can be applied to log data to uncover patterns, trends, and anomalies. The results of the various mining operations can generate actionable insights that help make better-informed decisions.

Machine learning is a concept that pairs well with observability. Anomaly detection models can analyze patterns in historical telemetry data such as metrics, logs, and traces to identify anomalies in system behavior. Anomalies may point toward services or functionalities within them that can be improved. Different models can be employed to predict future trends and events based on historical data, enabling proactive action and preemptive mitigation of potential issues such as resource shortages. Root Cause Analysis is the process of identifying relationships between components to find likely causes for performance degradation or failures within systems which is enabled by observability and often makes use of machine learning principles.

Since our solution is a proof of concept first and foremost, and currently meant to be deployed as an add-on to the target system, it may be wise or desirable to decouple the storage and visualization infrastructure from the production application. Enabling production-ready signal storage outside the production application can free up resources for either side of this equation. Being able to compare data from different deployments can give further unique insights into the behavior of the system.

The data collected by our proof-of-concept solution allows for a better understanding of the system but much more is left to unlock. We did not set out to build a solution to a specific problem, but instead to lay the groundwork for such topical observances. It is left to the users to build on this groundwork and to iterate on the generated and stored signals, the insights computed from them, and the visualizations to maximize the value gained.

8 Conclusion

We have shown that observability can be decomposed into four main problems that need to be addressed. These problems are signal generation, communication, storage, and visualization. However, understanding the problem is only the first step toward solving it. Further steps are: finding tools that are fit to solve these problems and jointly putting them to work. When looking for suitable tools, it is important to have clear goals defined and to stay focused on these goals.

Based on our findings, we think introducing a baseline of observability is a realistic undertaking for any organization or project. Even generic metrics can improve the understanding we have of a system. Furthermore, the existence of even a basic setup incentivizes iteration and improvement by removing the barrier of entrance. We felt that there were few resources addressing this by applying an analytical approach. So we set out to improve our understanding of the challenges of observability and to provide a comprehensible approach that can be used as a point of entry or guide. We verified our newly found understanding through a proof-of-concept implementation.

We think observing software systems is critical to understanding and improving them. Observability is more than just collecting data: the kind of data that is collected in the process enables informed decisions. Developers, operators, and managers alike stand to benefit from this.

A Tables

Criterion	Rating	Assessment
Tool Type		Emitter
Necessary Properties	4	Generate signals: ✓ Make signals available: ✓
Desirable Properties	2	No or little changes to service: added build dependencies
Extra Features	2	Default instrumentation for many common libraries
Dependencies	2	N/A
Complexity	1	Build definition needs to be adapted
Maintainability	2	N/A
Performance	2	No concern
Cost	4	Free with commercial support available
Documentation	4	Extensive documentation available from producer
Ecosystem	3	Integrations with other products; well integrated with Spring Boot
Community	3	4k+ Stars; 250+ Contributors
Support	4	Slack and GitHub Issues; Commercial support available
Security	2	No TLS out of the box however no concern
Reliability	2	No concern

Table A.1: Tool Selection Criteria for Micrometer

Criterion	Rating	Assessment
Tool Type		Emitter
Necessary Properties	4	Generate signals: ✓ Make signals available: ✓
Desirable Properties	3	No or little changes to service: Java agent JAR added
Extra Features	3	Default instrumentation for many common libraries; Configurable via Java Opts
Dependencies	2	N/A
Complexity	3	Enabled via configuration of runtime definition
Maintainability	2	up-to-date agent JAR needs to be made available to services
Performance	2	N/A
Cost	4	Free
Documentation	3	Extensive documentation from producer; sometimes feels incomplete
Ecosystem	4	Integration with many common libraries and first-level support for many products; Countless related projects
Community	3	Agent with 1.5k+ Stars on GitHub; 250+ Contributors; Part of bigger organization with many more users
Support	4	Slack; Mailing Lists; GitHub Issues
Security	4	TLS and client key auth available for OTLP modes
Reliability	2	No concern

Table A.2: Tool Selection Criteria for OpenTelemetry Agent

Criterion	Rating	Assessment
Tool Type		Emitter
Necessary Properties	2	Generate signals: ✓ Make signals available: if collected by additional service
Desirable Properties	4	No or little changes to service ✓
Extra Features	2	N/A
Dependencies	4	N/A
Complexity	4	No configuration required
Maintainability	4	Part of runtime environment
Performance	4	No further impact
Cost	4	Free
Documentation	4	Comprehensive explanations
Ecosystem	2	Tools for scraping from Docker Daemon
Community	4	Part of Docker; Official forums and Slack; other community-led channels
Support	2	Best effort and community support; Paid support for Docker available
Security	2	N/A
Reliability	2	No concern

Table A.3: Tool Selection Criteria for Docker Stats

Criterion	Rating	Assessment
Tool Type		Communication
Necessary Properties	4	Support for protocol on both sides: ✓
Desirable Properties	2	No additional components necessary: × Low overhead: ✓
Extra Features	4	Extensible via plugins; Pre-processing
Dependencies	4	None
Complexity	3	Single Container; Some configuration required
Maintainability	4	Stateless service
Performance	2	No concern
Cost	4	Free
Documentation	3	Lots of documentation but feels fragmented
Ecosystem	4	OpenTelemetry Ecosystem is big; Many plugins available
Community	4	Combined 5k+ GitHub Stars; 1k+ Contributors; However: Overlap between two repositories
Support	4	Slack; Mailing Lists; GitHub Issues
Security	4	TLS and client key auth available for OTLP modes
Reliability	2	No concern

Table A.4: Tool Selection Criteria for OpenTelemetry Collector

Criterion	Rating	Assessment
Tool Type		Communication
Necessary Properties	4	Support for protocol on both sides: ✓
Desirable Properties	2	No additional components necessary: × Low overhead: ✓
Extra Features	4	Extensible via plugins; Pre-processing
Dependencies	4	None
Complexity	3	Single Container; Some configuration required
Maintainability	4	Stateless service
Performance	2	No concern
Cost	4	Free
Documentation	4	Well documented; better curated
Ecosystem	4	Input and Output Plugins for many technologies; Well integrated with InfluxDB
Community	4	13k+ GitHub Stars; 1.2k Contributors to main project
Support	4	Community slack and forums for discussion/help; GitHub Issues for Issues and Feature Requests
Security	2	Authentication depending on Input/Output Plugin
Reliability	2	No concern

Table A.5: Tool Selection Criteria for Telegraf

Criterion	Rating	Assessment
Tool Type		Communication
Necessary Properties	2	Support for protocol on both sides: If overlap between Emitter and Storage support
Desirable Properties	4	No additional components necessary: ✓ Low overhead: ✓
Extra Features	2	N/A
Dependencies	4	No effort
Complexity	4	N/A
Maintainability	4	No effort
Performance	4	N/A
Cost	4	Free
Documentation	2	Depends on transport protocol used
Ecosystem	2	Depends on transport protocol used
Community	2	Depends on transport protocol used
Support	2	Depends on transport protocol used
Security	2	Depends on transport protocol used
Reliability	2	No concern

Table A.6: Tool Selection Criteria for Plain Communication Protocols

Criterion	Rating	Assessment
Tool Type		Metrics Storage
Necessary Properties	4	Persistent Storage: with additional long term storage backend
Desirable Properties	2	Reusable: × Low Maintenance Overhead: ✓ Scalable: limited; requires long term storage database and multiple instances for metrics ingestion
Extra Features	3	Web UI; Data Explorer; Query Builder
Dependencies	4	×
Complexity	4	Single container; little configuration necessary; Long term storage requires additional storage backend
Maintainability	3	Aimed at data acquisition and short term storage and therefore easy to maintain
Performance	4	Potential issues with high cardinality at large scale
Cost	4	Free & OSS
Documentation	4	Good documentation and many community guides
Ecosystem	4	Clients for many languages; exporters for many formats; integrations with many other products
Community	4	More than 50k stars on GitHub; more than 100 PRs and Issues; the only metrics database endorsed by Kubernetes; entire conventions focused on Prometheus
Support	4	Commercial support available; Community IRC; Slack; Discourse forums
Security	4	Basic auth and TLS for incoming connections; TLS for scrape target authentication
Reliability	4	Mostly positive; ideally operated with HA storage backend such as Mimir or Thanos

Table A.7: Tool Selection Criteria for Prometheus

Criterion	Rating	Assessment
Tool Type		Metrics Signal Storage
Necessary Properties	4	Persistent Storage: ✓
Desirable Properties	3	Reusable: ✓ Low Maintenance Overhead: ✓ Scalable: Only with paid license
Extra Features	4	Web UI; Data Explorer; Query Builder; Alerts Time Series Index can be flushed and rebuilt to purge out-of-date and corrupt information
Dependencies	4	×
Complexity	4	Single container
Maintainability	3	Data retention period can be set for data buckets
Performance	3	Not a concern. Faster than NoSQL databases like ElasticSearch; Cassandra; MongoDB. Slower than TimescaleDB at scale. Storage usage is good.
Cost	4	Free; Open Source with paid support and managed service available
Documentation	4	Interface; design and configuration documentation
Ecosystem	3	Various integrations
Community	4	28k GitHub Stars; 500+ Contributors; Dedicated forums; Slack
Support	3	Community forums; GitHub issues
Security	4	TLS; API Tokens
Reliability	1	Concerns because of ground-up development of complex system; many instances of corrupt data throughout community

Table A.8: Tool Selection Criteria for InfluxDB OSS v2

Criterion	Rating	Assessment
Tool Type		Metrics Signal Storage
Necessary Properties	4	Persistent Storage: ✓
Desirable Properties	3	Reusable: ✓ Low Maintenance Overhead: ✓ Scalable: with paid plan
Extra Features	4	Trace Signal Storage
Dependencies	4	×
Complexity	0	Several containers with different concerns; No easy access
Maintainability	1	CLI and REST API as only interfaces
Performance	4	Little data available; improved performance comparing to v2
Cost	4	Free OSS version or managed cluster
Documentation	4	Usage; administration; internals documented
Ecosystem	3	Similar programming interfaces as previous versions
Community	2	Dedicated forums
Support	4	Community forums; GitHub issues
Security	4	TLS; API Tokens; SSO & SAML available
Reliability	2	Concerns because of ground-up development of complex system; pre-release software

Table A.9: Tool Selection Criteria for InfluxDB v3

Criterion	Rating	Assessment
Tool Type		Trace Signal Storage
Necessary Properties	4	Persistent Storage: ✓
Desirable Properties	3	Reusable: × Low Maintenance Overhead: ✓ Scalable: ✓
Extra Features	3	Web UI; Data Explorer; Dependency Graph via external aggregation
Dependencies	3	Not required for working setup; external storage backend such as Apache Cassandra or Elastic-Search recommended for production workloads
Complexity	3	Several containers with different concerns; highly configurable; however: all-in-one solution available (not recommended for production workloads)
Maintainability	3	Maintainability largely depends on the chosen storage backend due to implementations of features such as data time-to-live
Performance	3	Jaeger only ingests trace signals; the heavy lifting is done by the storage backend which needs to be run in addition
Cost	4	Free OSS
Documentation	3	Usage; internals documented
Ecosystem	4	Implements messaging formats of Open-Telemetry framework which offers client libraries in many languages; sometimes referred to as the standard for cloud native distributed tracing
Community	4	More than 19k stars on GitHub; dedicated Slack channel
Support	3	GitHub issues; Slack channel
Security	2	TLS only available for Query Frontend; recommended security strategy involves reverse proxying collector instance(s) and terminating TLS at proxy
Reliability	4	No indicators for major risk

Table A.10: Tool Selection Criteria for Jaeger

Criterion	Rating	Assessment
Tool Type		Trace Signal Storage
Necessary Properties	4	Persistent Storage: ✓
Desirable Properties	3	Reusable: × Low Maintenance Overhead: ✓ Scalable: ✓
Extra Features	2	Metrics generator
Dependencies	4	×
Complexity	4	Grafana Tempo was designed for ease of operation; requires low maintenance and configuration effort
Maintainability	4	Offers automated maintenance and management features; data block TTL
Performance	4	Components can be scaled individually to account for any kind of load
Cost	4	Free OSS; paid SaaS offering available
Documentation	4	Configuration and architecture documented
Ecosystem	4	Compatible with many standards and tools; deeply integrated with other Grafana products
Community	3	3.6k stars on GitHub; more than 100 contributors
Support	4	Official forums; Slack channel; GitHub Issues
Security	4	TLS; regular security updates
Reliability	4	No indicators for major risk

Table A.11: Tool Selection Criteria for Grafana Tempo

Criterion	Rating	Assessment
Tool Type		Trace Signal Storage
Necessary Properties	4	Persistent Storage: ✓
Desirable Properties	3	Reusable: × Low Maintenance Overhead: ✓ Scalable: ✓
Extra Features	2	Web UI; Data Explorer; Dependency Graph via external aggregation
Dependencies	0	Storage Backend such as Apache Cassandra; ElasticSearch
Complexity	4	Can run as single container
Maintainability	4	No concerns
Performance	3	slower than e.g. Jaeger
Cost	4	Free & OSS
Documentation	4	Usage; administration; internals documented
Ecosystem	3	Integrations with several other technologies and products
Community	2	Gitter; GitHub
Support	2	GitHub issues
Security	1	N/A
Reliability	4	No concern

Table A.12: Tool Selection Criteria for Zipkin

Criterion	Rating	Assessment
Tool Type		Logs Signal Storage
Necessary Properties	4	Persistent Storage: ✓
Desirable Properties	3	Reusable: × Low Maintenance Overhead: ✓ Scalable: ✓
Extra Features	2	Metrics generator
Dependencies	3	Promtail or similar log shipper is not required but preferred
Complexity	4	Grafana Loki was designed for ease of operation; requires low maintenance and configuration effort
Maintainability	4	Offers automated maintenance and management features; data retention policies configurable
Performance	4	Components can be scaled individually to account for any kind of load
Cost	4	Free OSS; paid SaaS offering available
Documentation	4	Configuration and architecture documented
Ecosystem	4	Compatible with many standards and tools; deeply integrated with other Grafana products
Community	4	22k stars on GitHub; nearly 1000 contributors
Support	4	Official forums; Slack channel; GitHub Issues
Security	3	No authentication; secure through reverse proxy; regular security updates
Reliability	4	No indicators for major risk

Table A.13: Tool Selection Criteria for Grafana Loki

Criterion	Rating	Assessment
Tool Type		Logs Signal Storage
Necessary Properties	4	Persistent Storage: ✓
Desirable Properties	2	Reusable: ✓ Low Maintenance Overhead: × Scalable: ✓
Extra Features	3	Full ELK stack has countless features
Dependencies	1	Log shipper such as Logstash alongside ElasticSearch recommended; minimum 4GB of memory required
Complexity	1	Requires setting up indices; index rotations; designed to run in clusters
Maintainability	1	Complex setup and architecture
Performance	4	Single node clusters can ingest > 100k lines per second
Cost	3	Free OSS with paid extra features
Documentation	4	Extensive documentation of all aspects; setup; operation; architecture
Ecosystem	4	Tight integration with other Elastic Tools; many official client libraries; third-party clients and tools for numerous use cases
Community	4	67k stars on GitHub; nearly 2000 contributors
Support	4	GitHub Issues; official forums and Slack channel
Security	4	Security enabled by default; Token auth for APIs
Reliability	4	Optimized for relevant production workloads; focus on resilience and scalability

Table A.14: Tool Selection Criteria for ElasticSearch

Criterion	Rating	Assessment
Tool Type		Visualization
Necessary Properties	4	Compatibility with storage system: ✓ Capability to visualize as required: ✓
Desirable Properties	4	Reusable: ✓ Low maintenance overhead ✓
Extra Features	3	Alerts
Dependencies	4	×
Complexity	4	Single container
Maintainability	4	Dashboards; Alerting Rules; Datasources can be defined in flat config files
Performance	4	Query load is delegated to storage backends; Otherwise simple Web App
Cost	4	Free & OSS; Managed Cloud offering available
Documentation	4	Extensive documentation; guides
Ecosystem	4	Datasource plugins and integrations for many other backends
Community	4	60k+ Stars; 2k+ Contributors; Grafana Labs Community Slack
Support	4	GitHub Issues; Community Slack; Paid support available
Security	4	No concern
Reliability	4	No concern

Table A.15: Tool Selection Criteria for Grafana

Bibliography

- [1] Carlos Albuquerque et al. “Proactive monitoring design patterns for cloud-native applications”. In: *Proceedings of the 27th European Conference on Pattern Languages of Programs*. EuroPLop ’22: 27th European Conference on Pattern Languages of Programs. Irsee Germany: ACM, July 6, 2022, pp. 1–13. ISBN: 978-1-4503-9594-6. DOI: 10.1145/3551902.3551961. URL: <https://dl.acm.org/doi/10.1145/3551902.3551961> (visited on 11/13/2023).
- [2] Marcello Cinque, Raffaele Della Corte, and Antonio Pecchia. “Microservices Monitoring with Event Logs and Black Box Execution Tracing”. In: *IEEE Transactions on Services Computing* 15.1 (Jan. 1, 2022), pp. 294–307. ISSN: 1939-1374, 2372-0204. DOI: 10.1109/TSC.2019.2940009. URL: <https://ieeexplore.ieee.org/document/8826375/> (visited on 01/03/2024).
- [3] elastic. *elastic/elasticsearch: Free and Open, Distributed, RESTful Search Engine*. Elasticsearch Readme. Mar. 4, 2024. URL: <https://github.com/elastic/elasticsearch/blob/main/README.asciidoc> (visited on 05/10/2024).
- [4] Eric Evans. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional, 2004. URL: <https://books.google.com/books?hl=en&lr=&id=xColAAPGubgC&oi=fnd&pg=PR9&dq=domain+driven+design&ots=qdTHbjNP1s&sig=929PaM9L3d4JwWmU1M0nTNPgb5c> (visited on 06/13/2024).
- [5] Martin Fowler. *Microservices*. martinowler.com. URL: <https://martinfowler.com/articles/microservices.html> (visited on 06/11/2024).
- [6] T Hariprasad et al. “Software complexity analysis using halstead metrics”. In: *2017 International Conference on Trends in Electronics and Informatics (ICEI)*. 2017 International Conference on Trends in Electronics and Informatics (ICOEI). Tirunelveli: IEEE, May 2017, pp. 1109–1113. ISBN: 978-1-5090-

- 4257-9. DOI: 10.1109/IC0EI.2017.8300883. URL: <http://ieeexplore.ieee.org/document/8300883/> (visited on 06/10/2024).
- [7] Christoph Heger et al. "Application Performance Management: State of the Art and Challenges for the Future". In: *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering*. ICPE '17: ACM/SPEC International Conference on Performance Engineering. L'Aquila Italy: ACM, Apr. 17, 2017, pp. 429–432. ISBN: 978-1-4503-4404-3. DOI: 10.1145/3030207.3053674. URL: <https://dl.acm.org/doi/10.1145/3030207.3053674> (visited on 01/09/2024).
- [8] Ying Jiang, Na Zhang, and Zheng Ren. "Research on Intelligent Monitoring Scheme for Microservice Application Systems". In: *2020 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS)*. 2020 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS). Vientiane, Laos: IEEE, Jan. 2020, pp. 791–794. ISBN: 978-1-72816-698-8. DOI: 10.1109/ICITBS49701.2020.00173. URL: <https://ieeexplore.ieee.org/document/9109991/> (visited on 01/03/2024).
- [9] Joanna Kosińska et al. "Toward the Observability of Cloud-Native Applications: The Overview of the State-of-the-Art". In: *IEEE Access* 11 (2023), pp. 73036–73052. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2023.3281860. URL: <https://ieeexplore.ieee.org/document/10141603/> (visited on 01/08/2024).
- [10] Francesco Lomio, Sampsa Jurvansuu, and Davide Taibi. "Metrics selection for load monitoring of service-oriented system". In: *Proceedings of the 5th International Workshop on Machine Learning Techniques for Software Quality Evolution*. ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Athens Greece: ACM, Aug. 23, 2021, pp. 31–36. ISBN: 978-1-4503-8625-8. DOI: 10.1145/3472674.3473983. URL: <https://dl.acm.org/doi/10.1145/3472674.3473983> (visited on 11/13/2023).
- [11] T.J. McCabe. "A Complexity Measure". In: *IEEE Transactions on Software Engineering* SE-2.4 (Dec. 1976), pp. 308–320. ISSN: 0098-5589. DOI: 10.1109/TSE.1976.233837. URL: <http://ieeexplore.ieee.org/document/1702388/> (visited on 06/10/2024).

- [12] Fabio Pina et al. "Nonintrusive Monitoring of Microservice-Based Systems". In: *2018 IEEE 17th International Symposium on Network Computing and Applications (NCA)*. 2018 IEEE 17th International Symposium on Network Computing and Applications (NCA). Cambridge, MA: IEEE, Nov. 2018, pp. 1–8. ISBN: 978-1-5386-7659-2. DOI: 10.1109/NCA.2018.8548311. URL: <https://ieeexplore.ieee.org/document/8548311/> (visited on 01/03/2024).
- [13] Prabhat Ram et al. "Actionable Software Metrics: An Industrial Perspective". In: *Proceedings of the Evaluation and Assessment in Software Engineering. EASE '20: Evaluation and Assessment in Software Engineering*. Trondheim Norway: ACM, Apr. 15, 2020, pp. 240–249. ISBN: 978-1-4503-7731-7. DOI: 10.1145/3383219.3383244. URL: <https://dl.acm.org/doi/10.1145/3383219.3383244> (visited on 01/02/2024).
- [14] Chris Richardson. *Microservices Pattern: Microservice Architecture pattern*. microservices.io. URL: <http://microservices.io/patterns/microservices.html> (visited on 06/11/2024).
- [15] T.L. Saaty. *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. Advanced book program. McGraw-Hill International Book Company, 1980. ISBN: 978-0-07-054371-3. URL: <https://books.google.de/books?id=Xxi7AAAAIAAJ>.
- [16] Anas Shatnawi et al. "Cloudhealth: a model-driven approach to watch the health of cloud services". In: *Proceedings of the 1st International Workshop on Software Health. ICSE '18: 40th International Conference on Software Engineering*. Gothenburg Sweden: ACM, May 28, 2018, pp. 40–47. ISBN: 978-1-4503-5730-2. DOI: 10.1145/3194124.3194130. URL: <https://dl.acm.org/doi/10.1145/3194124.3194130> (visited on 11/13/2023).
- [17] Fotios Voutsas, John Violos, and Aris Leivadeas. "Filtering Alerts on Cloud Monitoring Systems". In: *2023 IEEE International Conference on Joint Cloud Computing (JCC)*. 2023 IEEE International Conference on Joint Cloud Computing (JCC). Athens, Greece: IEEE, July 2023, pp. 34–37. DOI: 10.1109/JCC59055.2023.00010. URL: <https://ieeexplore.ieee.org/document/10229586/> (visited on 11/12/2023).
- [18] Muhammad Waseem et al. "Design, monitoring, and testing of microservices systems: The practitioners' perspective". In: *Journal of Systems and Soft-*

Bibliography

ware 182 (Dec. 2021), p. 111061. ISSN: 01641212. DOI: 10.1016/j.jss.2021.111061. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0164121221001588> (visited on 01/03/2024).

Name: Claus Ruepp

Matrikelnummer: 905635

Erklärung

Ich erkläre, dass ich die Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet have.

Ulm, den

Claus Ruepp